Qiskit Fall Fest 2026 Hackathon - Advanced Track

Overview

Teams design and build a game whose mechanics are a real quantum error-detection task, with every score computed in Qiskit. The challenge is the error-detection track of "QFF 2026 Hackathon: Gamifying Error Detection, Mitigation and Correction", written by Wenyu Sun (third-year undergraduate in physics, McGill University) and provided by IBM Quantum for Qiskit Fall Fest 2026.

Who this track is for. For participants comfortable with quantum algorithms who want a deeper challenge. Participants getting started with quantum computing and Qiskit should take the Basic track (Geometry-aware quantum cloud challenge).

Background and problem statement

Problem statement. How can we engage more students in quantum error correction? Inspired by the 2D grid surface code and QGSS 2025 Lab 4, develop a game featuring elements of error correction that challenges players to optimize strategies against noise and errors. In this challenge the game is built around one specific technique: low-overhead error detection with spacetime codes.

The game must stay faithful to the physics. The aim is to present the ideas simply without losing their original meaning, so a game that only uses QEC vocabulary as decoration does not meet the brief.

Detection compared with mitigation and correction. Teams must be able to explain where their game sits:

- **Error detection** flags the errors that its ancilla checks can see and discards those shots; errors the checks do not cover go undetected. It does not repair the state. This is the subject of this challenge.
- **Error mitigation** leaves the noisy hardware as it is and corrects the *estimate* of an observable in post-processing (ZNE, PEC, twirling, readout correction), at the cost of extra circuits and shots.
- **Error correction** encodes one logical qubit in many physical qubits, measures stabilizers repeatedly, and uses a decoder to infer and undo errors while the computation runs.

Short history.

1. 1994: Shor's factoring algorithm shows a polynomial-time quantum algorithm, but qubits are fragile.
2. Shor's 9-qubit code encodes one logical qubit in nine physical qubits and corrects both a bit flip (X) and a phase flip (Z).
3. CSS (Calderbank-Shor-Steane) codes refine the idea with fewer qubits; stabilizer codes generalize it.
4. Kitaev's topological codes lead to the surface code, which needs only nearest-neighbour checks on a 2D grid.

5. qLDPC codes reduce physical-qubit overhead. Example: the bivariate bicycle code [[144, 12, 12]] stores 12 logical qubits in 144 physical qubits at distance 12; its checks reach qubits that are not nearest neighbours.

Learning outcomes.

1. Distinguish between error detection, error mitigation and error correction.
2. Break down the main ideas of quantum error correction (stabilizers, syndromes, surface and qLDPC codes).
3. Understand what makes practical quantum computing difficult.
4. Build and innovate on top of existing methods.

Guiding question: what is fault tolerance, and what will we need to achieve it?

The challenge: error detection with spacetime codes

Build a game in which the player decides how to protect a circuit with spacetime-code checks, and is scored on the resulting fidelity computed in Qiskit.

Tool. The qiskit-paulice add-on implements low-overhead error detection with spacetime codes using coherent Pauli checks. Related background: the coherent-parity-check (CPC) code entry in the Error Correction Zoo. Visual reference: spacetime codes of Clifford circuits.

Access and cost. qiskit-paulice is free and open source (Apache 2.0) and installs with `pip install qiskit-paulice`. Everything in this challenge runs locally on Qiskit Aer; no IBM Quantum account or paid plan is needed. The official tutorial calls `QiskitRuntimeService` only to fetch a real backend's calibration data. A fake backend (`NoiseModel.from_backend(FakeXxx(), layout=...)`) or a synthetic noise model (`NoiseModel(gate_noise=0.01)`) works the same way.

Clifford-only input. `add_pauli_checks` accepts Clifford circuits only (gates cx, cz, h, s, sdg, sx, sxdg, x, z, and rz at multiples of π/2). A circuit containing `t`, or `rz(π/4)`, is rejected with an error.

Pipeline the game should expose:

1. **Payload circuit**: the data qubits and the logic being run.
2. **Spacetime checks**: controlled-Pauli (X, Y or Z) checks inserted into the circuit.
3. **Ancilla readout**: syndrome measurement.
4. **Post-selection filter**: shots with a non-trivial syndrome are discarded.

Factors that affect fidelity: circuit depth, number of T gates, and where the spacetime checks are placed.

Do: calculate the fidelity of the circuit. Compare the fidelity of a Clifford-only circuit with a T-doped circuit.

Goal: determine the best fidelity rate, meaning the best trade-off between post-selected fidelity and the fraction of shots kept.

Required steps

1. **Build the payloads.** One Clifford-only circuit and a family of T-doped versions of it, with the number of T gates varied (for example 0, 1, 2, 4, …). State the circuit width and depth.
2. **Fix the reference.** State how the ideal output is computed (stabilizer simulation for the Clifford circuit; statevector simulation for small T-doped circuits) and which fidelity measure you use, with a one-line justification.
3. **Add checks with qiskit-paulice.** Add checks to the Clifford-only circuit, varying how many checks are added (the function returns one circuit per number of checks) and which target qubits they attach to. The add-on cannot take the T-doped circuits, so for those, either report fidelity without checks, or design your own checks by hand. A hand-made check is valid across a T gate only if its Pauli acts as I or Z on that qubit at that point, because T is diagonal and commutes with Z.
4. **Run under noise.** Use the noise model or fake backend announced by the organizers, so that results are comparable across teams.
5. **Measure.** For every configuration, record the fidelity without checks, the post-selected fidelity with checks, the fraction of shots kept, and the added cost (ancilla qubits, two-qubit gates, depth).
6. **Compare.** Show how fidelity and kept-shot fraction change with T count, circuit depth and check location. Identify the configuration with the best trade-off between post-selected fidelity and kept-shot fraction, and explain why it wins.

A post-selected fidelity is always reported together with its kept-shot fraction. A strategy that keeps 2% of shots is not comparable to one that keeps 80%.

Building the game

The game's rules must correspond to the decisions in the steps above, and its score must come from the computed numbers, not from hand-set values. One possible mechanic: the player has a budget of ancilla qubits and chooses where in a Clifford circuit to place checks; the score combines fidelity and kept-shot fraction. Teams are free to design their own mechanic.

Example of a QEC game for inspiration: A Game of Surface Codes

Litinski's A Game of Surface Codes (Quantum, 2019) turns surface-code computation into a board game. It is about error correction, not detection, but it shows how a QEC concept can become a set of game rules.

- The board is a grid of tiles. Each tile is a block of physical qubits; each patch is one logical qubit and may occupy one or more tiles.
- Each patch edge is either an X edge or a Z edge.
- Every operation has a time cost: initialization costs 0, a measurement may cost 1.
- If two patches have adjacent edges of the same type, their product (for example $Z \otimes Z$) can be measured through the region between them.
- Multi-qubit Pauli products can be measured using auxiliary (ancilla) regions.

What it teaches: space-time trade-offs. More qubits allow faster computation; fewer qubits make it slower. A game idea built on it: for a given layout and time budget N, perform S operations in N time steps, as space-efficiently as possible.

Submission

Upload everything to a single **public** GitHub repository, then submit **only the repository link**. The repository must contain:

1. **Code**: complete, runnable source code
2. **Recorded presentation**: a video walkthrough of your approach and results
3. **Report**: a written report describing the problem, method and findings

Rules, the code of conduct and all other details are on the hackathon website. For questions, write to qff@iitj.ac.in with "Hackathon" in the subject line.

Judging criteria

The weights and guiding questions come from the original prompt. Presentation carries the most weight (40%), so teams must be able to explain and justify their design, not only build it.

Criterion	Weight	Judges' question	What scores high
Presentation	40%	Can the participants explain their methods succinctly and justify their design choices?	Clear mapping from each game rule to the physics; honest limitations; the recorded presentation explains the approach and justifies the design choices.
Effect	30%	Does it provide conceptual breakthroughs and further insights?	The game shows a non-obvious trade-off (fidelity against kept shots, T count against check placement) backed by numbers.
Methods	20%	Is it creative and does it accurately incorporate QEC elements?	Correct physics; scores computed from Qiskit results, not hand-set; original mechanic.
Qiskit	10%	How did the participants integrate Qiskit in their project (for example, add-ons	qiskit-paulice and primitives used as intended; current Qiskit APIs;

Criterion	Weight	Judges' question	What scores high
		used)?	reproducible notebook.

Scoring scale. Each judge scores each criterion from 0 to 10. Team score = 4 × Presentation + 3 × Effect + 2 × Methods + 1 × Qiskit, giving a maximum of 100. The final score is the mean across judges.

Resources

Starter notebooks

- QGSS 2025 Lab 4 (the inspiration for this prompt)
- QGSS 2026 Lab 3

Error detection with spacetime codes

- qiskit-paulice documentation
- Low-overhead error detection with spacetime codes (tutorial)
- Coherent-parity-check (CPC) code (Error Correction Zoo)
- Researchers demonstrate quantum advantage through trusted quantum computation (IBM blog, includes the spacetime-code visualization)
- Sampling hard circuits with verifiably high fidelity

Theory

- Foundations of quantum error correction (IBM Quantum Learning course)
- What's the difference between error suppression, error mitigation and error correction?
- What is fault-tolerant quantum computing?
- qLDPC codes for quantum error correction
- Realizing repeated quantum error correction in a distance-three surface code
- Quantum error correction below the surface code threshold

Games built on quantum error correction (for inspiration)

- A Game of Surface Codes (Litinski, Quantum 3, 128, 2019) and its PennyLane demo
- Introducing lattice surgery
- Quantum Poker: a game for quantum computers suitable for benchmarking error mitigation techniques on NISQ devices
- Game-theoretic discovery of quantum error-correcting codes through Nash equilibria
- sneakyweasel/genetic-quantum-correction (GitHub)

Original prompt: "QFF 2026 Hackathon: Gamifying Error Detection, Mitigation and Correction" by Wenyu Sun, third-year undergraduate in physics, McGill University. Provided by IBM Quantum for Qiskit Fall Fest 2026.